

## TP numéro 6 Gestion des processus et services

### Prérequis et environnement

Vous allez travailler dans la machine virtuelle **Ubuntu Server**.

### Exercice 1

L'objectif est d'observer la différence de comportement entre un arrêt propre et un arrêt brutal.

1. Lancez une commande longue en arrière-plan (ex : `sleep 1000 &`).

**Exécution de la commande dans le terminal :**

```
1 $ sleep 1000 &
2 [1] 45678
```

2. Identifiez son PID à l'aide de la commande `ps` et vérifiez son état dans `/proc/[PID]/status`.

**Utilisation de `ps` et lecture du pseudo-système de fichiers `/proc` :**

```
1 $ ps aux | grep sleep
2 $ cat /proc/45678/status | grep State
3 # Affiche : State: S (sleeping)
```

3. **Expérimentation SIGTERM :**

— Créez un script nommé `handler_demo.sh` avec le contenu suivant :

```
1 #!/bin/bash
2
3 # routine de fermeture
4 cleanup() {
5     echo "[$(date)] Nettoyage avant sortie..." >> /tmp/demo.log
6     exit 0
7 }
8
9 #trap : commande shell qui intercepte des signaux ou événements
   système
10 trap cleanup SIGTERM
11
12 echo "PID du script : $$"
13 while true; do
14     echo "En cours d'exécution..."
15     sleep 5
16 done
```

- Envoyez un SIGTERM (15) et observez si le processus peut exécuter la routine de fermeture (`kill -15 <PID>`).

**Le processus reçoit SIGTERM, exécute la fonction `cleanup` (qui écrit dans `/tmp/demo.log`) puis se termine proprement.**

**Le fichier `/tmp/demo.log` contient une ligne avec la date et le message "Nettoyage avant sortie..."**

#### 4. Expérimentation SIGKILL :

- Relancez le processus et envoyez un SIGKILL (9) (`kill -9 <PID>`).
- Pourquoi est-il impossible pour le processus d'intercepter ce signal ? Quelles sont les conséquences potentielles sur une base de données en production ?

**SIGKILL est non interceptable. Le noyau détruit immédiatement le processus sans lui laisser la possibilité d'exécuter son handler `cleanup`. Aucun nettoyage n'a lieu. Conséquences en production : les buffers en RAM ne sont pas écrits sur le disque (perte de données pour une BDD), et les sockets réseau peuvent rester bloquées dans l'état `TIME_WAIT`.**

## Exercice 2

1. Lancez `sleep 300` en arrière-plan avec `nohup` : `nohup sleep 300 &`
2. Fermez le terminal et ouvrez-en un nouveau. Vérifiez si `sleep` est toujours actif avec `ps aux | grep sleep`. Notez votre observation.

**Le processus `sleep` est toujours en cours d'exécution.**

**`nohup` ignore le signal `SIGHUP` envoyé par le shell lors de la fermeture du terminal. Le processus survit donc à la déconnexion.**

## Exercice 3

1. Créez un script `/usr/local/bin/gi_ia_service.sh` avec le contenu suivant :

```
1 #!/bin/bash
2 while true; do
3     echo "Service actif à $(date)" >> /var/tmp/gi_ia_service.log
4     sleep 10
5 done
```

Rendez-le exécutable.

2. Créez un fichier de service systemd `/etc/systemd/system/gi_ia_service.service` (avec `sudo`) contenant :

```
1 [Unit]
2 Description=Mon service de démonstration
3 After=network.target
4
5 [Service]
6 Type=simple
7 ExecStart=/usr/local/bin/gi_ia_service.sh
8 Restart=on-failure
9
10 # Sandboxing
11 User=nobody
```

```

12 Group=nogroup
13 ProtectSystem=strict
14 PrivateTmp=true
15 NoNewPrivileges=true
16 ProtectHome=true
17
18 [Install]
19 WantedBy=multi-user.target

```

3. Activez et démarrez le service :

```

1 sudo systemctl daemon-reload
2 sudo systemctl enable gi_ia_service.service
3 sudo systemctl start gi_ia_service.service

```

4. Vérifiez que le service fonctionne : `systemctl status gi_ia_service`. Quel est l'état ? Consultez le fichier de log `/var/tmp/gi_ia_service.log`.

### Solution

- `systemctl status gi_ia_service` doit afficher `active (running)`.
- La directive `PrivateTmp=true` demande à `systemd` de créer un espace isolé (un "namespace") pour les répertoires `/tmp` et `/var/tmp` spécifiquement pour ce service.  
Ainsi, lorsque votre script fait `echo ... >> /var/tmp/gi_ia_service.log`, il n'écrit pas dans le `/var/tmp` global de votre machine, mais dans un dossier temporaire privé qui ressemble à ceci :  
`/var/tmp/systemd-private-<boot-id>-gi_ia_service.service-<random>/tmp/gi_ia_service.log`

5. **Test du sandboxing** : Essayez d'accéder au répertoire `/home` depuis l'intérieur du service. Pour cela, modifiez temporairement le script pour y inclure `ls /home` et redémarrez le service. Que se passe-t-il ? (Indice : consultez les logs avec `journalctl -u gi_ia_service`).

### Solution

- La commande `ls /home` échoue avec une erreur de type :  
`ls: cannot open directory '/home': Permission denied.`
- Les logs `journalctl -u gi_ia_service` montrent l'erreur.
- **Explication** : La directive `ProtectHome=true` protège l'accès au répertoire `/home`.

6. Ajoutez une limitation de mémoire à 50 Mo dans la section `[Service]` : `MemoryMax=50M`. Redémarrez le service. Vérifiez avec `systemctl show gi_ia_service | grep MemoryCurrent` (peut nécessiter quelques secondes). Que se passe-t-il si le service dépasse cette limite ?

### Solution

- La commande `systemctl show mon_service | grep MemoryCurrent` affiche la consommation mémoire actuelle du cgroup du service.
- Si le service tente d'allouer plus de 50 Mo, le noyau invoque l'OOM killer (Out-Of-Memory) spécifique au cgroup et tue le processus. `systemd` peut ensuite le redémarrer grâce à `Restart=on-failure`.

## Exercice 4 (optionnel)

On souhaite automatiser une sauvegarde du répertoire `/etc` vers `/var/backups/system`.

1. **Méthode Cron** : Configurez une tâche cron pour l'utilisateur root qui effectue cette sauvegarde chaque lundi à 03h00. Utilisez des chemins absolus.

**Édition via `sudo crontab -e`. Il faut échapper le % dans cron car il est interprété comme un retour à la ligne s'il n'est pas protégé.**

```
1 0 3 * * 1 /usr/bin/tar -czf /var/backups/system/etc_$(date +%F).tgz /etc
```

2. **Méthode Systemd Timer** :

— Créez un script `/usr/local/bin/backup_etc.sh` (pensez au `chmod +x`).

**Contenu du script `/usr/local/bin/backup_etc.sh` :**

```
1  #!/bin/bash
2
3  # Arrête le script immédiatement si une commande échoue
4  set -e
5
6  # --- Définition des variables ---
7  # On utilise un chemin relatif pour SOURCE car on va utiliser "
   tar -C /"
8  SOURCE="/etc"
9  DEST_DIR="/var/backups/system"
10 DATE=$(date +%F) # Format AAAA-MM-JJ (ex: 2026-04-19)
11 ARCHIVE="${DEST_DIR}/etc_${DATE}.tar.gz"
12
13 # Préparation ---
14 # systemd capturera ces "echo" et les enverra directement dans
   journalctl
15 echo "[INFO] Début de la sauvegarde de /${SOURCE}..."
16
17 # Création du répertoire de destination s'il n'existe pas
18 mkdir -p "${DEST_DIR}"
19
20 # sauvegarde ---
21 # L'option -C / permet à tar de se placer à la racine avant d'
   archiver "etc".
22 # Cela évite le warning "tar: Removing leading `/' from member
   names"
23 tar -czf "${ARCHIVE}" -C / "${SOURCE}"
24
25 echo "[OK] Archive créée : ${ARCHIVE}"
26
27 # Nettoyage : on supprime les archives de plus de 30 jours pour
   éviter de saturer le disque
28 echo "[INFO] Nettoyage des sauvegardes de plus de 30 jours..."
29 find "${DEST_DIR}" -type f -name "etc_*.tar.gz" -mtime +30 -
   delete
30
31 echo "[OK] Opération de sauvegarde terminée avec succès."
```

— Créez un fichier unité `backup-etc.service` de type oneshot.

#### Contenu de `/etc/systemd/system/backup-etc.service` :

```
1 [Unit]
2 Description=Sauvegarde hebdomadaire de /etc
3
4 [Service]
5 Type=oneshot
6 ExecStart=/usr/local/bin/backup_etc.sh
```

- Créez le fichier `backup-etc.timer` associé pour une exécution hebdomadaire.

#### Contenu de `/etc/systemd/system/backup-etc.timer` :

```
1 [Unit]
2 Description=Timer pour la sauvegarde de /etc
3
4 [Timer]
5 OnCalendar=Mon *--* 03:00:00
6 Persistent=true
7
8 [Install]
9 WantedBy=timers.target
```

3. **Analyse comparative** : Redémarrez votre machine. Expliquez comment la directive `Persistent=true` du timer permet de rattraper la sauvegarde manquée, contrairement à `cron`.

La directive `Persistent=true` écrit un fichier d'état sur le disque (souvent dans `/var/lib/systemd/timers`). Au démarrage, `systemd` compare cette date avec l'heure de la dernière exécution prévue. Si une échéance a été manquée car le serveur était éteint, `systemd` déclenche le service immédiatement. `cron` quant à lui, va ignorer l'exécution manquée.